

Otimização Eficiente da Taxa de Aprendizado por Polling para Redes Neurais

Luiz Henrique • José Ronaldo

Centro de Informática (CIn) — Universidade Federal de Pernambuco (UFPE)
Recife, Brasil

Motivação

Por que a taxa de aprendizado é tão crítica?

A taxa de aprendizado (LR) é um dos hiperparâmetros mais influentes no treinamento de redes neurais.

LR alta demais → divergência ou oscilação. LR baixa demais → convergência lenta ou mínimos ruins.

Abordagens tradicionais (decaimento por fórmula, cosine annealing, Adam, RMSProp) ajustam a LR com heurísticas fixas predefinidas — "aproveitamento" (exploitation): nunca saem do que a fórmula permite.

Tan et al. propõem um paradigma diferente: polling — testar várias LRs candidatas a cada passo e escolher a que mais melhora o desempenho no lote atual.

É uma forma de "exploração" (exploration) guiada por desempenho medido, sem nenhuma fórmula de decaimento.

Objetivos deste Trabalho

1. Replicar o Método de Polling de Tan et al.

- Cenário original: ANN simples + MNIST, LRs candidatas entre 0,01 e 0,05
- Novo cenário: CNN treinada com SGD puro no CIFAR-10, candidatas cobrindo 5 ordens de magnitude ($1e-5$ a $1e-1$)

2. Propor e avaliar uma extensão: o Método de Polling Eficiente

- Resolve o principal problema do método original: custo computacional alto
- Faz polling sob demanda em vez de a cada lote

Comparação das duas abordagens com uma linha de base de SGD com LR fixa, ao longo de 150 épocas.

Método 1 — Polling

Replicado de Tan et al.

A cada lote de treinamento:

- 1. Calcula-se o gradiente g a partir dos pesos atuais θ (forward + backward, uma única vez)
- 2. Tira-se um snapshot dos pesos e do estado do otimizador
- 3. Para cada uma das $N=5$ LRs candidatas $C = \{1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$: aplica-se um passo experimental $\hat{\theta}_i = \theta - lr_i \cdot g$ e mede-se a acurácia no próprio lote
- 4. Escolhe-se a candidata com maior acurácia no lote (empate $\rightarrow$ menor LR)
- 5. Os pesos vencedores tornam-se o ponto de partida do próximo lote

Critério de seleção: acurácia do lote (não a perda).

Custo: 5 atualizações de peso experimentais por lote $\rightarrow$ mais que triplica o tempo de treino.

Método 2 — Polling Eficiente

Contribuição proposta neste trabalho

Mesmo mecanismo de seleção do Método 1 — mas faz polling sob demanda.

(a) Backoff exponencial: intervalo K entre execuções de polling

- Escolha igual à anterior $\rightarrow K$ dobra (até $K_{\max} = 64$)
- Escolha diferente $\rightarrow K$ volta a 1 (novo polling já no próximo lote)
- Lotes entre polls: 1 passo de SGD comum "às cegas", com a última LR vencedora

(b) Proteção contra divergência em 2 níveis

- Nível 1 — pico (prevenção): perda do lote $> \gamma = 3 \times \text{EMA}(\text{perda}, \beta=0,9) \rightarrow$ força polling imediato
- Nível 2 — checkpoint/rollback (recuperação): perda não-finita ou $> 2 \cdot \ln(10) \approx 4,61 \rightarrow$ reverte ao último checkpoint

4 constantes ($K_{\max}=64$, $\gamma=3$, $\beta=0,9$, limiar $\approx 4,61$) — nenhuma precisou de ajuste fino.

Por que a Proteção é Necessária

Sem o mecanismo de proteção, os passos "às cegas" não têm nenhuma validação antes de serem aplicados.

Execução preliminar SEM proteção:

- Um único passo "às cegas" com $LR = 1e-1$ por volta da época 33 (na fase de LR alta)
- $\rightarrow$ divergência numérica: perda virou NaN
- $\rightarrow$ o treino NUNCA se recuperou durante as 117 épocas restantes

Conclusão: a proteção em 2 níveis não é opcional — sem ela, o Polling Eficiente seria inviável.

Configuração Experimental

Dataset: CIFAR-10 — 50.000 treino / 10.000 teste, 10 classes, 32×32 RGB (45.000 treino / 5.000 validação após split)

Modelo: SimpleCIFAR10CNN — CNN simples de 5 camadas, ~557.898 parâmetros, sem batch norm e sem dropout (o otimizador é a única fonte de adaptação em estudo)

Otimizador base: SGD puro, sem momento e sem weight decay, em todos os métodos

Batch size 64, 150 épocas, LR base 1e-3 — 704 lotes/época × 150 = 105.600 lotes no total

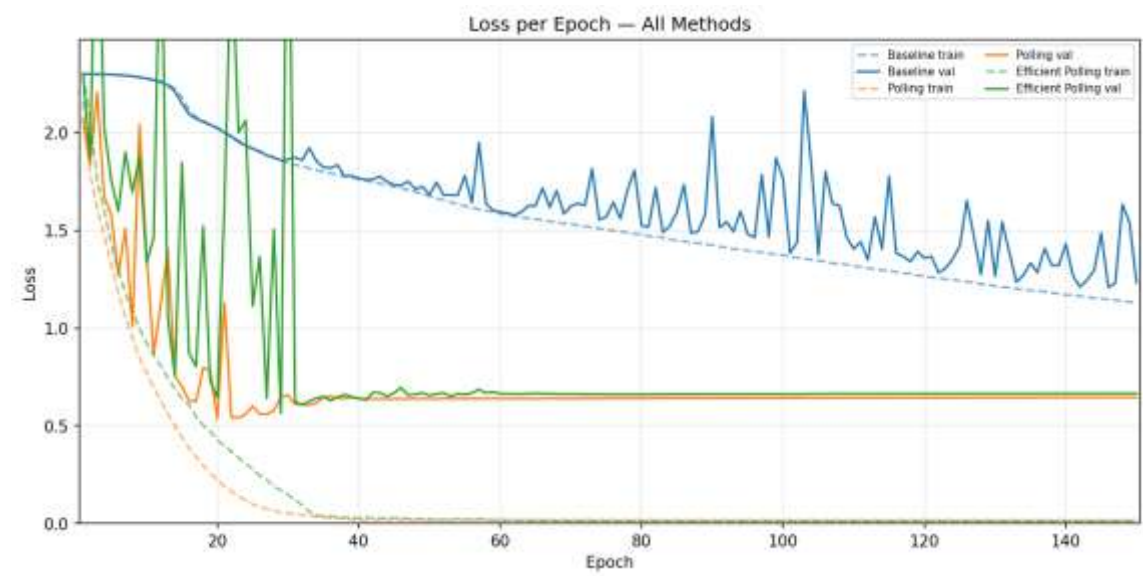
Mesma seed (42) para inicialização, embaralhamento e split nos 3 métodos — única diferença é a estratégia de LR

Hardware: 1× NVIDIA RTX 5070 (12GB)

Resultados — Desempenho

150 épocas completas

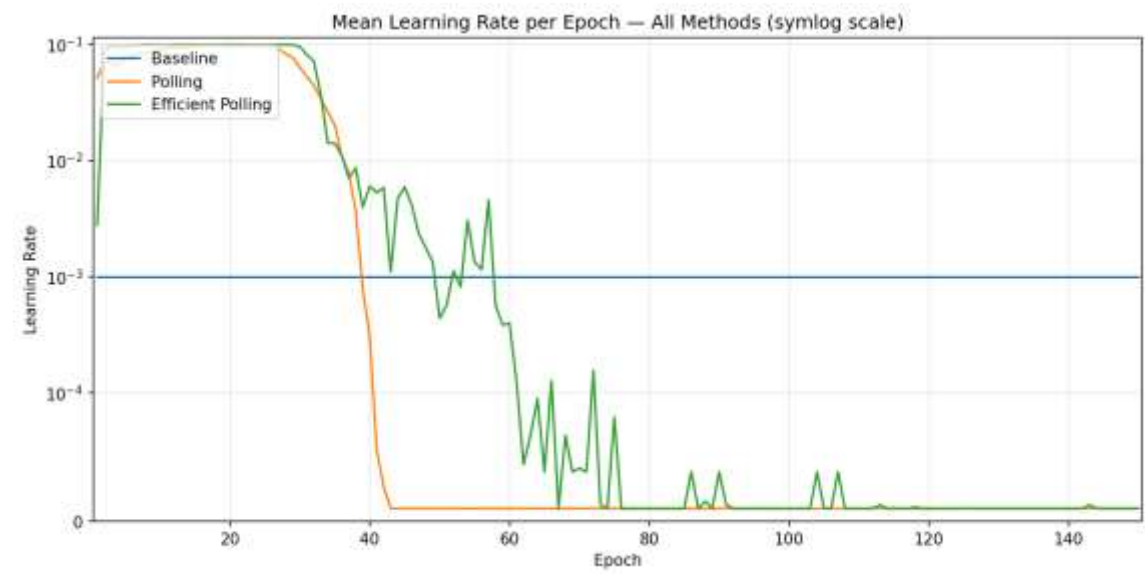
Método	Melhor Val.	Acur. Teste	Perda Teste
Linha de base (SGD fixo 1e-3)	57,28%	56,70%	1,2155
Polling (replicado)	84,65%	83,99%	0,6832
Polling Eficiente (proposto)	85,07%	83,93%	0,7319



Resultados — Custo Computacional

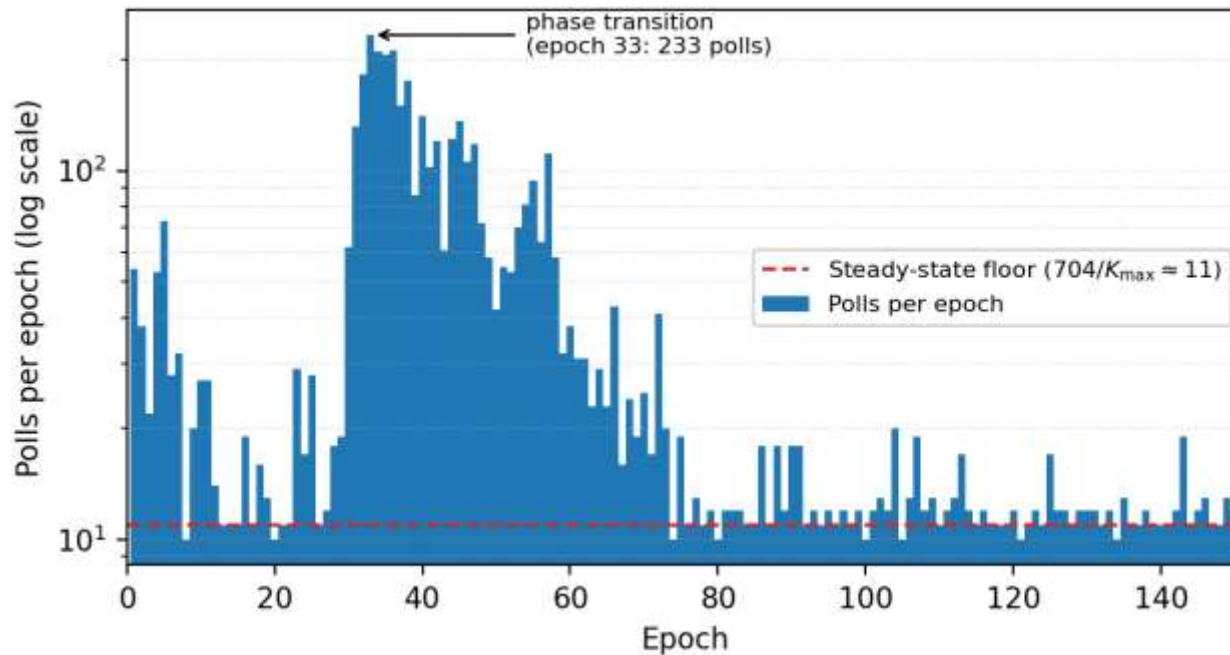
150 épocas, 105.600 lotes

Método	% Lotes c/ Polling	Passos Otim.	s/Época
Linha de base	0%	105.600	2,50
Polling (replicado)	100%	528.000	9,10
Polling Eficiente	5,05%	132.285	2,79



Onde o Custo se Concentra

Polls por época — Polling Eficiente



Regime estável: ~ 11 polls/época (backoff saturado em $K_{\max}=64$)

Pico isolado: 233 polls na época 33

— Exatamente quando a LR transita de alta para baixa

Total: apenas 5.337 de 105.600 lotes com polling (5,05%)

Nenhum rollback (Nível 2) foi acionado — só o Nível 1 (pico) foi necessário

Obrigado!

Perguntas?